Agent Effort under Resource Budgets: Conditional Allocation and Two Small-Model Case Studies

Zewen Liu
Independent Researcher
173538952630163.com

Abstract

An optimal resource allocation under an assumed success function need not describe the behavior of a tool-using language model. We examine that gap using an illustrative five-axis allocation calculation and retained records from eight authored programming tasks. The allocation calculation searches 8,820 tuples and yields a modeled token-cost ratio of 3.44 against a fixed-middle setting at at least matched modeled success; its gains, token prices, and unconstrained action combinations are assumptions, not measured deployment benefits. In two Qwen3 AWQ case studies, each with 672 retained execution records, the same five coupled budget settings yield different success profiles: the 8B records peak at 0.9375 at the low budget and reach 0.7500 at the maximum, whereas the 14B records peak at 1.0000 in the middle and reach 0.8750 at the maximum. These are descriptive rates on eight tasks, with two nested menus and identical repeated outcomes, rather than independent samples establishing a general decline. A protocol audit identifies further interpretation limits: success requires an explicit passing test within budget, the supplied scaffold repeats the original starter and truncates history, and structural probes are not visual understanding. Missing generated code and an unresolved runtime-script fingerprint prevent a complete execution audit. The contribution is a reproducible accounting of the retained outcomes and the assumptions separating an allocation model from a bounded agent evaluation.

1 Introduction

A tool-using agent spends resources on several activities: generating text, taking turns, invoking local tools, requesting external information, and inspecting observations. A resource model can assign prices and gains to those activities and then select the best allocation under a budget. That optimization is only as informative as its success function and feasibility constraints. More permitted actions also need not cause a particular agent to use them productively.

This paper examines the distinction through a small case study. An illustrative allocation model assumes saturating gains along four agent-resource axes and borrows three reasoning-effort anchors from the DeepSeek-V4.1-Flash report(DeepSeek-AI, 2026). Its grid optimum is compared with recorded execution outcomes from two Qwen3 AWQ configurations on eight authored programming tasks. We do not run DeepSeek-V4.1-Flash, transfer its measured capability curve to Qwen, or propose an evaluated online routing policy.

The retained data support a narrower question: *what do these budget-conditioned completion records measure, and which conclusions survive an audit of their units and protocol?* The two model

configurations have different point-estimate profiles. Repeated records do not supply new task outcomes, and success is a test-passing event rather than a retrospective audit of every final program. The supplied scaffold also has memory and observation conventions that complicate a capability interpretation. These features matter before fitting marginal resource gains or asserting deployment savings.

We contribute three bounded results. First, we independently recompute the conditional allocation comparisons and distinguish measured latency primitives from assumed success and token prices. Second, we report the model records at the task level, including the dependence between menus and repeated executions. Third, we audit the supplied harness and record provenance, identifying test-selection, context-accounting, and execution-binding limits. This is a descriptive study of retained records, not a new benchmark-scale evaluation or a general law of agent scaling.

2 Related work and positioning

Adaptive and controllable test-time computation are established research areas(Alomrani et al., 2025). Self-consistency allocates compute to sampling and aggregation(Wang et al., 2023); CoBa considers generation, verification, and stopping(Zhou et al., 2026); thought-level beam search explores partial reasoning trajectories(Yang et al., 2026). These approaches motivate explicit allocation decisions, but their evaluations do not validate our assumed success function.

Budget-Aware Tool Use already provides a joint token-and-tool cost metric and studies budget-aware scaling(Liu et al., 2025). EcoAgent-Bench evaluates economic decisions under priced actions and explicit budgets(Wu et al., 2026). We therefore make no claim to first unify agent costs or discover that budget awareness differs from economical behavior. Our distinct material is the accounting and protocol audit of this particular small task suite and its retained records.

The DeepSeek-V4.1-Flash report conditions training on a requested effort level and discusses effort-dependent token penalties and empirical tradeoffs(DeepSeek-AI, 2026). Importantly, its Appendix B.2 already discusses non-monotonic performance under different scaffolds, and Appendix C limits its theoretical approximation. Our case studies neither contradict a universal monotonicity claim from that report nor reproduce its model results. The programming setting is also much smaller than repository-level benchmarks such as SWE-bench(Jimenez et al., 2024): our tasks are hand-authored functions with a few tests, not real issue-resolution tasks.

3 Evidence and analysis design

3.1 Evidence layers

We distinguish four evidence layers. E1/E1b contain host-local latency samples for 18 primitives. E2 is a deterministic numerical allocation calculation. E3 is a rule-based reference repairer that writes supplied correct code and executes a prescribed action schedule. H4 contains two sets of records labeled as real model-driven executions, using Qwen3-8B-AWQ and Qwen3-14B-AWQ with thinking disabled. Each H4 file has 672 records: 240 coupled-budget records and 432 single-axis-sweep records. The two H4 files contain environment metadata, full file fingerprints for 13 checkpoint files each, action summaries, timing, budgets, and success flags.

The H4 metadata identify Python 3.10.12, vLLM 0.30.0, and PyTorch 2.13.0+cu130. Checkpoint fingerprints improve identification of the recorded model files, but the weights are not redistributed. The retained script fingerprint does not match the supplied harness snapshot; the exact runtime

Table 1: Coupled-budget outcomes. Each cell has 48 recorded executions but only eight authored tasks, each tested under two menus and three repeats. Repeats have identical success outcomes. Rates also equal the mean of the eight within-task rates; no independent-sample intervals are implied. E3 uses prescribed schedules and supplied correct code; H4 uses model-selected actions.

Budget	Steps	Tools	Obs. chars	Probes	E3	8B	14B
L0	2	1	200	0	0.0000	0.0000	0.0000
L1	4	3	600	1	0.5000	0.9375	0.8125
L2	8	6	1,500	2	1.0000	0.8750	1.0000
L3	16	12	4,000	4	1.0000	0.8125	0.8750
L4	32	24	12,000	8	1.0000	0.7500	0.8750

snapshot has not been recovered. Consequently, the protocol description below reports the supplied implementation, while the numerical results describe retained records. We do not claim that inspecting the current code completely verifies the historical executions.

3.2 Tasks, budgets, and menus

The same eight authored tasks are reused across budgets, menus, and model configurations: grouped sum, JSON round trip, stable deduplication, binary search, word frequency, matrix transpose, interval merging, and top-k. Each has a starter function, a reference solution, and three unit tests. The supplied H4 implementation presents a task specification derived from the test contract and the starter text. The tests are hidden through the ordinary action interface, not withheld from the experiment author or protected against arbitrary executed Python. Passing this small test suite does not establish correctness for all inputs or every part of the verbal contract.

Table 1 lists the five coupled budgets. The historical field `max_context_chars` counts cumulative read/probe observation characters; it is not a full model-context or token limit. The probe is a structural summary consisting of four string lengths, including the test-script length. It neither presents an image nor measures visual reasoning. We call it a structural probe even though the historical field uses `vision`.

H4 offers two menus: read/write/test, and read/probe/write/test. The model selects actions and their order. E3 instead forces either read/probe/write/test or probe/read/probe/write/test and writes the supplied reference solution. E3 shares tasks, budget values, and the grading function with H4, but its schedules and information differ. It is an accounting control, not a strictly matched policy-replacement experiment.

3.3 Units and recomputation

Our post hoc analysis uses the six retained result files listed in the ancillary bundle. The generator validates coverage, unique H4 task/arm/menu/repeat keys, declared budgets, real-versus-stub flags, and selected inherited aggregates. It rederives latency summaries, task rates, and E2 comparisons without invoking a model or network service.

An H4 coupled-budget cell has 48 executions: eight tasks, two menus, and three repeats. Across all conditions within each model file, the three repeats have identical success outcomes and retained action summaries. They are repeated executions, not additional independent task draws. The two menus are nested within the same task. We average within task and then equally across eight tasks; because the design is balanced, this reproduces the pooled rates. E3 has the same multiplicities but

does not retain repeat identifiers, so we validate three records per task/budget/schedule rather than inventing repeat IDs.

The tasks were authored and reused, not sampled from a defined population. We therefore report descriptive differences and leave-one-task-out ranges, with no population p-values or confidence intervals. Those ranges diagnose sensitivity to task composition; they are not uncertainty intervals. Historical record-level intervals, task-menu independence tests, and fitted marginal-gain intervals are retained as history but are not adopted as inferential evidence in this revision. The analysis is not an external timestamped preregistration.

4 Conditional allocation calculation

4.1 Assumptions and objective

An allocation is $a = (r, s, t, q, v)$: a reasoning-effort coordinate, steps, local calls, search calls, and probe operations. E2 assigns the modeled token cost

$$C(a) = sL(r) + 1000s + 200(t + q) + 400q + 200v, \qquad L(r) = 1200 \exp\left(\frac{\log 2.5}{75}(r - 25)\right). \quad (1)$$

The extra 400 search tokens are in addition to the 200 per-call term. All absolute token prices, including $L(25) = 1200$, are modeling assumptions; they are not measured Qwen token counts. The report provides mean Pass@1 anchors of 0.671 and 0.763 across eight reasoning-intensive benchmarks at requested effort 25 and 100, and an approximate output-token multiplier of 2.5. Borrowing these anchors does not validate the interpolation or transfer them to our agent tasks.

With $s_0 = 5000$, define

$$w(r) = \frac{\log(1 + (r - 1)/s_0) - \log(1 + 24/s_0)}{\log(1 + 99/s_0) - \log(1 + 24/s_0)}, \quad (2)$$

$$A(r) = 0.671 + (0.763 - 0.671)w(r), \quad (3)$$

$$G(a) = 1 - \prod_{x \in \{s,t,q,v\}} [1 - g_x(1 - e^{-x/h_x})], \quad (4)$$

$$P(a) = \min\{1,\ 1 - [1 - A(r)G(a)](1 - 0.35)\}. \quad (5)$$

The gains are $(0.30, 0.25, 0.28, 0.20)$ and references $(10, 6, 4, 4)$ in step/tool/search/probe order. These quantities are declared assumptions, not estimated marginal benefits from the H4 records. The implementation's front-loading routine reaches its parameter boundary: $w(80) = 0.7348$, rather than its intended target 0.76. We retain that actual calculation and do not call it an exact fitted source curve.

The reasoning levels are 0, 25, 50, 75, and 100. Step levels are 1, 2, 4, 8, 12, 20, and 32; local-call levels are 0, 1, 2, 4, 8, 16, and 24. Search and probe levels are 0, 1, 2, 4, 8, and 16. Their product is 8,820 allocations. The zero reasoning coordinate is an extrapolation outside the report's advertised 1–100 interface, not a valid requested effort tier established by that report. The optimization computes

$$P^*(B) = \max_{a \in \mathcal{A}: C(a) \le B} P(a), \qquad C^*(u) = \min_{a \in \mathcal{A}: P(a) \ge u} C(a). \quad (6)$$

This is exhaustive finite-grid optimization with known assumed outcomes. It does not learn a deployed controller.

Table 2: Conditional allocation calculation. Modeled success and token costs are algebraic outputs. Minimum cost means the least grid cost attaining at least the baseline modeled success; discrete-grid overshoot is allowed. Ratios are baseline cost divided by that minimum, not measured deployment savings.

Baseline	Modeled success	Modeled tokens	Minimum cost	Ratio
All maximum	0.6906	145,600	145,600	1.00
Fixed middle	0.5320	23,429	6,810	3.44
Reasoning maximum	0.5478	34,400	7,610	4.52
Agent axes maximum	0.6633	101,717	49,600	2.05

4.2 Results and interpretation

Table 2 is independently recalculated from Equations 1–5. The fixed-middle tuple is $(50, 8, 4, 2, 2)$. Reasoning maximum replaces its first coordinate with 100; agent-axes maximum replaces its remaining coordinates with their grid maxima. The 3.44, 4.52, and 2.05 ratios compare those settings with the cheapest grid point reaching at least their respective modeled success, not necessarily equal success. At the all-maximum success, the ratio is 1.00: no modeled cost saving is established at that endpoint.

These comparisons have structural advantages for the optimizer. Every baseline belongs to its feasible grid, so selecting the optimum cannot do worse under the same assumed objective and budget. The success function increases with each coordinate by construction. Neither property is an empirical discovery about LLM agents. Furthermore, the grid treats coordinates as independently allocatable: it can assign many calls to a single step without an executable action-dependency constraint. For example, its 20,000-token optimum is $(100, 1, 16, 16, 16)$, with modeled success 0.6305. That tuple is not a feasible one-action-per-turn H4 trajectory. E2 is thus an illustrative resource relaxation, not a validated deployment frontier.

E1/E1b contribute measurements only to the auxiliary latency accounting, not to the token-cost ratios in Table 2. On one host, a shell read/verify primitive averages 7.091 ms, an external HTTP GET averages 2233.753 ms, and PNG encoding/decoding averages 177.341 ms. The HTTP GET is not a measured search-agent operation, and the PNG codec is not a visual-model inference. All 30 screenshot-capture attempts fail on the headless host; all 40 deliberately unreachable network attempts fail. Table 4 reports every primitive, including failures. These local measurements do not establish portable latency ratios or actual agent end-to-end costs.

We do not use the inherited latency bootstrap to certify success-function assumptions. Nor do we adopt its sensitivity summary as a matched robustness test: the original routine changes parameters for the optimized model while comparing with an unchanged reference baseline. A valid deployment claim would require recalibrated gains, feasible trajectories, measured token consumption, and a budget-matched evaluated controller.

5 Retained execution outcomes

5.1 Coupled budgets and task composition

The E3 reference repairer reaches rates $0, 0.5, 1, 1, 1$ across L0–L4. Its low-budget split follows its prescribed schedules: the shorter schedule completes, while the probe-heavy schedule exhausts

Table 3: Within-task success fractions (two menus averaged; repeats do not add new outcomes). The last column gives the L4 minus L1 difference. These eight tasks are fixed authored examples, not a random sample of programming tasks.

	8B			14B		
Task	L1	L4	Difference	L1	L4	Difference
t01	1.00	1.00	0.00	1.00	1.00	0.00
t02	1.00	0.00	-1.00	1.00	1.00	0.00
t03	1.00	1.00	0.00	0.50	1.00	0.50
t04	1.00	1.00	0.00	1.00	1.00	0.00
t05	1.00	1.00	0.00	0.00	1.00	1.00
t06	0.50	0.50	0.00	1.00	1.00	0.00
t07	1.00	1.00	0.00	1.00	0.00	-1.00
t08	1.00	0.50	-0.50	1.00	1.00	0.00

its permitted probes before writing and testing. This outcome demonstrates the known schedule/constraint interaction with supplied correct code; it does not estimate an LLM capability curve.

The H4 8B rates are 0, 0.9375, 0.8750, 0.8125, and 0.7500. The corresponding 14B rates are 0, 0.8125, 1, 0.8750, and 0.8750 (Table 1). These are finite-suite profiles. The L1-to-L4 change is -0.1875 for 8B and $+0.0625$ for 14B. The latter does not reproduce the former's low-to-max direction. Both have a lower maximum-budget rate than their own observed peak, but these data do not establish a population decline or refute a general scaling law.

Table 3 makes the dependence visible. For 8B, two tasks worsen in the menu-averaged L1-to-L4 contrast, none improve, and six tie. For 14B, two improve, one worsens, and five tie. Removing one task at a time gives difference ranges $[-0.2143, -0.0714]$ for 8B and $[-0.0714, 0.2143]$ for 14B. In particular, even the sign of the 14B average is sensitive to which task is omitted. Persistence across the 8B leave-one-out set is a description of these eight tasks, not independent evidence of a universal degradation effect.

5.2 Recorded actions and what success counts

At L4, all four failed 8B task-menu conditions in the first repeat have 32 retained actions: one read and 31 writes, with no test. Their three repeats have the same summaries. In contrast, the two failed 14B conditions both concern interval merging and include multiple test attempts: one has 17 writes, nine tests, and six reads; the other has 24 writes, five tests, and three reads. Across all coupled-budget executions, mean tests per episode change from 0.9375 to 0.7500 for 8B and from 1.1875 to 2.0000 for 14B between L1 and L4.

These action counts do not prove a common verification-collapse mechanism. The supplied H4 implementation declares success when an explicit test returns a passing exit code within budget. It does not uniformly grade every final file after a budget failure. A correct but untested file can therefore fail this outcome variable, just as an incorrect file can. The failed 8B traces establish absence of a retained test event, not the correctness or incorrectness of their overwritten programs. Generated code and final files were not retained, so this distinction cannot be resolved retrospectively. Test invocation and success are also linked by the outcome definition; their association is not independent causal evidence.

All L0 model records fail, but the two-step/one-tool cap is not a proof that no policy could pass: a policy could in principle write and test in two actions. The observed agents take a read/write path and exhaust steps without a test. This is a policy-and-protocol observation rather than a universal budget floor.

5.3 Single-axis sweeps

Table 5 reports every single-axis rate, without fitting marginal gains. The swept caps are steps, tool calls, and structural probes; the other caps remain at middle values and may still bind. Only the probe-enabled menu is used in these sweeps. No reasoning-effort coordinate or search action is varied, and the structural probe does not measure visual understanding. The present records therefore cannot identify five universal gains or validate the gains assumed in E2. Boundary-hitting fits and repeated outcomes in the historical analysis do not repair that measurement gap.

6 Protocol audit and reproducibility limits

Memory and budget visibility. The supplied H4 code shows the budget to the model each turn. It also repeats the original starter, stores a write in history only as its character count, and truncates serialized history to its last 3000 characters without a truncation marker. Current code is available again only if the agent reads the file. A separate state field preserves action names, but it cannot reconstruct discarded code or observations. These conventions leave history drift, repeated-starter effects, budget-prompt effects, and hard resource limits as competing explanations. Budget visibility does not rule them out. The exported historical observation record incorrectly leaves history truncation as `null`; the source audit documents this discrepancy rather than treating it as evidence of no truncation.

Counter and grading semantics. In the supplied harness, the step counter increments before the next-step limit check. On a steps failure it can therefore equal the cap plus one even though that next model call did not execute. The retained list describes recorded action/attempt summaries; this sentinel count alone does not count completed actions. A write consumes a step but no tool-call count; read, probe, and test consume tool calls. The observation-character cap counts only read/probe strings and is checked after the action. It does not count the specification, original starter, serialized history, generated code, or actual tokenizer length.

Tests run in a real Python subprocess and normal grading checks successful unit tests, but the outer record uses the process return code. Generated Python runs in the same directory as the test script. No operating-system isolation or tamper-resistant grading boundary is established. We do not claim that any retained run exploited that weakness; missing code prevents a complete grading-integrity audit. The outcome is most precisely a recorded budget-limited test-pass event.

Runtime identity and missing traces. Both retained H4 files report script SHA-256 `3326b513...`, while the supplied snapshot hashes to `7e2d56e7...`; full values and the search record are in the evidence note. The supplied source is useful for inspection but cannot be certified as the exact historical runtime. Model file hashes do not solve this script-version gap. Complete model responses, per-turn prompts, written code, stop reasons, and final workspaces are absent. Our recomputation verifies arithmetic and retained-grid consistency; it does not independently verify that every historical execution followed the supplied code.

Scope and stronger follow-up. Eight authored tasks with three tests each cannot establish repository-level effectiveness, general resource-scaling laws, or cross-model superiority. Both model configurations reuse the same tasks, and the coupled sweep simultaneously changes several caps and their prompt text. The study does not isolate a pure compute intervention. A stronger follow-up should freeze the actual runtime snapshot before execution, retain complete prompts/responses and programs, grade final files independently of the chosen test action, strengthen grader isolation, define executable allocation constraints, and evaluate a controller on held-out tasks with task-cluster uncertainty. Those are proposed repairs, not experiments completed by this revision.

Recomputation package. The ancillary archive `anc/agent-effort-records.zip` contains the six retained result files, supplied experiment code, the new standard-library recomputation script, provenance notes, and member hashes. Running `python3 scripts/recompute_arxiv_r1.py` regenerates the descriptive JSON and five tables without inference calls. The archive is intended to accompany the article upon public posting; no public identifier is yet available. Checkpoint weights and the third-party report are linked rather than redistributed. Re-executing the experiments requires appropriate model weights, a compatible runtime, and a newly frozen harness; it is distinct from recomputing these retained results.

7 Conclusion

An allocation optimum under assumed gains and a budget-conditioned agent completion rate answer different questions. The conditional calculation yields cost ratios within its algebra and unconstrained grid. The retained model records instead show task- and model-specific profiles, nested repeated measurements, and action-selection effects entangled with test-based scoring. The supplied scaffold and missing runtime snapshot further limit explanation. Reporting that separation permits reproducible descriptive analysis without turning simulation savings or a small-suite point estimate into a demonstrated deployment benefit.

Broader impact

Resource accounting can help make agent evaluation more transparent, but interpreting proxy observations as capability or efficiency guarantees can mislead deployment decisions. This study uses authored programming tasks and retained experimental records, with no human-subject data. It establishes no production safety or cost-saving guarantee.

Use of generative AI tools

Codex assisted with manuscript revision, recomputation code, and internal review. It is not an author. No new model experiment was conducted for this revision. Tool-assisted checks and internal reviewer agents do not constitute external peer review or verification of the missing historical execution traces.

References

Mohammad Ali Alomrani, Yingxue Zhang, Derek Li, et al. Reasoning on a budget: A survey of adaptive and controllable test-time compute in LLMs. *arXiv preprint arXiv:2507.02076*, 2025. URL `https://arxiv.org/abs/2507.02076`.

DeepSeek-AI. Deepseek-v4.1-flash: Pushing the limits of KV cache compression. Technical report, DeepSeek-AI, 2026. URL https://arxiv.org/abs/2609.19969. Version 1, September 17, 2026. Sections 5.1.4 and 5.3.2; Appendices B.2 and C.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? *arXiv preprint arXiv:2310.06770*, 2024. URL https://arxiv.org/abs/2310.06770.

Tengxiao Liu, Zifeng Wang, Jin Miao, I-Hung Hsu, Jun Yan, Jiefeng Chen, Rujun Han, et al. Budget-aware tool use enables effective agent scaling. *arXiv preprint arXiv:2511.17006*, 2025. URL https://arxiv.org/abs/2511.17006.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2023. URL https://arxiv.org/abs/2203.11171.

Jie Wu, Ming Gong, Feixiang Cheng, and Qinqin Zhao. Ecoagent-bench: Evaluating economic decision-making in budget-constrained LLM agents. *arXiv preprint arXiv:2608.05519*, 2026. URL https://arxiv.org/abs/2608.05519.

Lijie Yang, Hongyin Luo, Jiawei Zhao, Tri Dao, and Ravi Netravali. Thought-level beam search for reasoning. *arXiv preprint arXiv:2608.08020*, 2026. URL https://arxiv.org/abs/2608.08020.

Yan Zhou, Yue Ouyang, Kaiyang Zheng, and Suncheng Xiang. Coba: Cost-effective test-time scaling via compute-balanced routing. *arXiv preprint arXiv:2608.07424*, 2026. URL https://arxiv.org/abs/2608.07424.

A Complete host-local latency accounting

Table 4: All retained latency primitives. Means and standard deviations include failed attempts; counts of successful invocations are separate. Units are milliseconds. These are sequential samples on one host, without a portability or independent-sample interval claim.

Primitive ID	Attempts	Successful	Mean ms	SD ms
P1_image_codec_png	60	60	177.341	12.675
P2_screenshot_capture	30	0	0.075	0.057
P3_test_run_small	30	30	58.245	3.445
P4_local_http_search	60	60	1.018	1.084
P5_patch_apply	60	60	2.264	0.205
P6_json_tool_call	60	60	33.982	14.429
T1_shell_simple	120	120	1.305	0.116
T2_shell_command	90	90	7.129	0.912
T3_python_exec	60	60	30.365	3.025
T4_file_read_small	200	200	0.033	0.008
T5_file_read_medium	60	60	0.052	0.025
T6_file_write_atomic	90	90	0.512	0.264
T7_glob_scan_200	120	120	0.101	0.065
T8_net_reachable	40	40	2233.753	2155.258
T9_net_unreachable	40	0	961.820	1914.169
T10_cpu_kernel	60	60	1.941	7.792
T11_process_spawn_tree	45	45	38.335	8.275
T12_shell_read_verify	60	60	7.091	1.312

The complete primitive names in Table 4 preserve the historical identifiers for recomputation. The reachable-network primitive issues HTTP GET requests to known reachable pages; the unreachable case deliberately targets a reserved invalid domain. The loopback-search primitive queries a local index. The PNG primitive encodes and decodes a random RGB frame. None is a substitute for measuring an actual internet search agent or vision-language inference. Failure-attempt timings are included in the all-attempt mean and standard deviation.

B Complete single-axis sweep accounting

Table 5: Single-axis sweep rates under the probe-enabled menu. Every cell has eight tasks and three repeats (24 executions); other caps remain fixed at steps=8, tools=4, observation characters=1500, probes=2. Those caps can bind. These rates do not isolate a universal marginal resource gain.

Swept cap	Value	8B rate	14B rate
Steps	2	0.0000	0.0000
Steps	4	0.7500	0.8750
Steps	8	0.7500	0.8750
Steps	16	0.8750	1.0000
Steps	32	0.7500	1.0000
Tools	0	0.0000	0.0000
Tools	1	0.0000	0.0000
Tools	2	0.6250	0.7500
Tools	4	0.7500	0.8750
Tools	8	0.6250	1.0000
Tools	16	0.6250	1.0000
Tools	24	0.6250	1.0000
Structural probes	0	0.8750	0.7500
Structural probes	1	0.6250	1.0000
Structural probes	2	0.7500	0.8750
Structural probes	4	0.7500	0.8750
Structural probes	8	0.7500	0.8750
Structural probes	16	0.7500	0.8750

The controls are caps rather than realized expenditures. Zero permitted test tool calls structurally prevents the required test-pass event in the supplied H4 protocol, but increasing a cap does not force a productive action. The displayed profiles include all swept levels; no level is removed for poor fit or non-monotonicity.